

Roteiro detalhado das simulacoes Ondas e Cores

Este material foi reescrito em tom mais academico para servir como texto de apoio a um artigo ou relato de experiencia em ensino de Fisica. O objetivo e descrever, com precisao conceitual e clareza metodologica, o funcionamento das duas versoes do projeto: **Ondas_e_Cores.py**, destinada ao uso local em PC com Matplotlib, e **app_web.py**, destinada ao navegador com Dash e Plotly.

Em ambas as implementacoes, o nucleo conceitual permanece o mesmo: construcao de duas ondas por sintese de Fourier, calculo de envelopes por transformada de Hilbert, comparacao entre envelopes como criterio de semelhanca global e interpretacao cromatica de frequencias visiveis em ambiente RGB. A versao web, por sua vez, introduz diferencas de arquitetura, interacao e avaliacao que ampliam o alcance pedagogico do simulador.

Problema didatico central: investigar de que modo a composicao espectral de um sinal pode ser manipulada pelo estudante, observada no dominio do tempo e reinterpretada por meio de envelopes de amplitude e de representacoes cromaticas, articulando conteudos de ondulatoria, analise de sinais e percepcao de cor.

Versao PC: Ondas_e_Cores.py

Funcoes da versao PC

Versao web: app_web.py

Funcoes da versao web

Versao PC - Ondas_e_Cores.py

Nesta versao, a interface principal e uma figura do Matplotlib. A janela central concentra os graficos e, ao redor dela, aparecem sliders verticais, botoes teoricos e controles da otimizacao. Cada botao superior abre uma nova janela tematica.

1. O que existe em cada tela da versao PC

Tela principal: Simulador

Grafico Onda 1

Mostra o sinal temporal da Onda 1, calculado pela soma de senos associados as frequencias entre 400 THz e 800 THz. O eixo horizontal esta em femtossegundos e o eixo vertical mostra a amplitude total da onda.

Grafico Onda 2

Repete a mesma logica para a Onda 2. Isso permite comparar duas sinteses diferentes construidas a partir do mesmo conjunto de frequencias disponiveis.

Grafico Envelopes de Hilbert

Exibe os envelopes das duas ondas. Em vez de mostrar cada oscilacao rapida, esse grafico destaca a variacao lenta da amplitude instantanea de cada sinal.

Grafico Diferenca entre Envelopes

Plota a funcao delta entre envelope 1 e envelope 2. Quando essa curva se aproxima de zero ao longo do tempo, temos um indicio forte de proximidade entre os dois sinais em termos de modulacao de amplitude.

Resumo rapido

Apresenta o pico absoluto de cada onda e a diferenca maxima entre envelopes. Esse bloco ajuda a transformar a leitura visual em um criterio numerico.

Colunas de sliders

As amplitudes de cada frequencia sao controladas por sliders verticais. A coluna da esquerda pertence a Onda 1 e a da direita a Onda 2.

Janela Fourier (Antes)

Aberta pelo botao Fourier (Antes). Mostra as equacoes da sintese correspondente ao estado anterior a otimizacao. Se ainda nao houve otimizacao, usa o estado atual.

Janela Fourier (Depois)

Aberta pelo botao Fourier (Depois). Mostra as equacoes montadas a partir das amplitudes resultantes da ultima otimizacao executada.

Janela Envelopes Hilbert

Aberta pelo botao Envelopes Hilbert. Reune a definicao matematica do envelope complexo, a formula do modulo, a interpretacao fisica e a explicacao do criterio de minimizacao usado na otimizacao.

Janela Lab RGB

Aberta pelo botao Lab RGB. Mostra uma barra de cores organizada por frequencia e quatro cartoes de cor: Onda 1 antes, Onda 1 depois, Onda 2 antes e Onda 2 depois. O usuario pode alternar entre dois modos: RGB visual e Paleta em RGB.

Janela Exercicios

Na versao PC, essa janela ainda e uma estrutura de preparacao. Ela explica a ideia pedagogica da futura aba de exercicios, mas nao implementa o fluxo completo que existe na versao web.

2. Tela "Simulador": graficos, informacoes e construcao

Grafico Onda 1 e Grafico Onda 2

Cada grafico e construido pela funcao **_generate_wave**, que soma varias componentes senoidais. Cada componente tem a forma:

$$y_n(t) = A_n \cdot \sin(2.\pi.f_n.t)$$

A onda total e a soma de todas essas componentes:

$$y(t) = \text{soma de } A_n \cdot \sin(2.\pi.f_n.t)$$

No codigo, isso aparece como uma soma sobre os pares amplitude-frequencia. Assim, os graficos Onda 1 e Onda 2 nao sao desenhos independentes: eles sao a visualizacao direta da sintese de Fourier definida pelos sliders.

Grafico Envelopes de Hilbert

Esse grafico e construido a partir do sinal no tempo. Primeiro calcula-se a transformada de Hilbert do sinal. Depois monta-se o modulo do sinal analitico. Em linguagem matematica:

$$z(t) = x(t) + j.H\{x(t)\}$$

$$A(t) = |z(t)| = \text{raiz quadrada de } [x(t)^2 + H\{x(t)\}^2]$$

Didaticamente, isso produz uma curva mais lenta, que acompanha o "contorno" da oscilacao. A onda pode oscilar muito rapidamente, mas o envelope destaca como a energia aparente do sinal cresce e decresce ao longo do tempo.

Grafico Diferenca entre Envelopes

O grafico Diferenca nao plota a diferenca entre as ondas brutas, e sim entre os seus envelopes:

$$\Delta(t) = \text{envelope}_1(t) - \text{envelope}_2(t)$$

Isso e pedagogicamente importante porque duas ondas podem ter fases ou detalhes de oscilacao diferentes e, ainda assim, compartilhar um comportamento global de

amplitude muito proximo. Quando a curva delta fica pequena em modulo, o programa conclui que as duas ondas estao mais proximas em termos de envelope.

3. Interatividade na versao PC

- Os sliders alteram, em tempo real, as amplitudes de cada frequencia das duas ondas.
- Os botoes superiores abrem janelas teoricas ou janelas auxiliares de interpretacao.
- O controle Suavizacao Hilbert muda a janela da media movel aplicada depois do calculo do modulo do envelope.
- O controle Max. iteracoes define o teto computacional da busca automatica.
- O controle delta f otimizado limita o quanto a otimizacao pode se afastar das frequencias ja ativas.
- O botao OTIMIZAR dispara uma busca numerica automatica.
- O botao ZERAR reinicia amplitudes, memoria de antes/depois e mensagem de status.
- O botao Grade liga ou desliga uma malha visual de apoio para entender o layout da figura.

4. Sintese de Fourier: como os graficos Onda 1 e Onda 2 sao gerados

Na simulacao, a sintese de Fourier aparece de forma concreta: cada slider escolhe o coeficiente de uma componente senoidal. Em vez de partir da formula abstrata e depois chegar ao grafico, o estudante faz o caminho inverso: altera coeficientes, ve a forma de onda mudar e depois consulta a equacao correspondente.

Onda 1: $y_1(t) = \text{soma de } A_n^{(1)} \cdot \sin(2\pi \cdot f_n \cdot t)$

Onda 2: $y_2(t) = \text{soma de } A_n^{(2)} \cdot \sin(2\pi \cdot f_n \cdot t)$

Cada termo traz uma contribuicao parcial. Quando muitos termos sao somados, a forma resultante pode ficar bastante diferente de uma senoide simples. O valor didatico disso e mostrar que sinais complexos podem ser descritos como superposicao de ondas elementares.

5. Explicacao didatica da sintese de Fourier

Leitura conceitual: Fourier ensina que um sinal complexo pode ser reconstruído pela soma de oscilações simples. Aqui, cada frequência entre 400 THz e 800 THz funciona como um "tijolo" da construção do sinal.

Se uma amplitude é zero, aquela frequência não participa. Se a amplitude é positiva ou negativa, ela entra com maior ou menor peso, inclusive invertendo fase no caso do sinal negativo. O estudante percebe que mudar poucos coeficientes já altera bastante a forma de onda final.

6. Envelopes de Hilbert: significado matemático e didático

O envelope de Hilbert é uma forma de separar a oscilação rápida do comportamento mais global da amplitude. Em vez de perguntar "qual é o valor instantâneo do sinal?", pergunta-se "qual é o tamanho instantâneo da oscilação?".

- $x(t)$: sinal real original.
- $H\{x(t)\}$: transformada de Hilbert, que produz uma versão defasada do sinal.
- $z(t)$: sinal analítico, combinando parte real e parte em quadratura.
- $|z(t)|$: módulo do sinal analítico, interpretado como amplitude instantânea.

O programa ainda permite uma suavização adicional por média móvel. Isso não faz parte da definição matemática do envelope de Hilbert, mas ajuda didaticamente a enxergar um contorno mais limpo e menos serrilhado.

7. Como os envelopes criam uma curva suavizada

Primeiro, o módulo do sinal analítico já remove a alternância positiva-negativa rápida da onda. Depois, quando a suavização adicional está ligada, aplica-se uma média móvel de janela ímpar sobre o envelope calculado. Isso filtra pequenas irregularidades locais e deixa mais evidente a tendência geral da amplitude.

Em sala de aula, essa etapa é útil porque ajuda o estudante a separar duas escalas de leitura: a oscilação microscópica da senoide e a variação macroscópica do envelope.

8. Diferença entre envelopes e confirmação de proximidade

O critério usado pela otimização não é deixar as ondas idênticas ponto a ponto, mas sim aproximar seus envelopes. Isso é medido pela maior distância absoluta entre as

curvas de envelope ao longo do tempo:

$$\text{erro} = \max | \text{envelope}_1(t) - \text{envelope}_2(t) |$$

Se esse erro maximo diminui, o programa conclui que as ondas estao mais proximas no sentido de modulacao de amplitude. Essa e uma comparacao robusta porque ignora parte das oscilacoes finas e foca no comportamento global.

9. Otimizacao: como as alteracoes sao feitas automaticamente

A versao PC usa **scipy.optimize.minimize** com o metodo L-BFGS-B. O vetor de busca contem apenas as amplitudes liberadas pela mascara de otimizacao. Frequencias fora da janela autorizada permanecem fixas.

- O programa salva o estado inicial em **amps_before**.
- Cria uma mascara booleana com **_build_optimization_mask**.
- Monta um vetor unico contendo amplitudes livres de Onda 1 e Onda 2.
- Define a funcao custo como a diferenca maxima entre envelopes.
- Em cada iteracao, reconstrui as duas ondas com **_merge_optimized_values**.
- Interrompe quando a meta de diferenca atinge 0.25 ou quando o limite de iteracoes e alcançado.

Didaticamente, isso mostra um exemplo excelente de problema inverso: em vez de escolher manualmente todos os coeficientes, o aluno define uma meta global e o algoritmo procura uma combinacao de amplitudes que aproxime os envelopes.

10. Tela Lab RGB: frequencia e cor

O Lab RGB converte a frequencia em um comprimento de onda aproximado e, a partir dele, monta uma cor RGB aproximada para a faixa visivel. Aqui ha duas leituras complementares:

Modo RGB visual

Calcula um RGB medio emitido pelas componentes ativas e depois renormaliza esse vetor para que o maior canal vire referencia visual. Isso facilita a comparacao na tela, mesmo quando a intensidade absoluta seria muito baixa.

Modo Paleta em RGB

Mantem a leitura mais diretamente ligada a ordem das frequencias visiveis. A exibicao continua sendo RGB, porque o monitor continua limitado aos tres canais, mas a paleta organiza melhor a intuicao espectral.

Nos cartoes, o texto distingue duas coisas: **Emite**, isto e, o RGB medio calculado a partir das frequencias ativas; e **Mostra**, isto e, a cor efetivamente usada na interface para representar aquele resultado.

11. Algo a mais que vale destacar na versao PC

- O simulador trabalha com frequencias tipicas da faixa visivel, o que conecta diretamente sintese de onda e cor.
- A janela de Fourier ajuda a passar da representacao visual para a simbolica.
- A janela de Hilbert ajuda a passar da observacao intuitiva para a linguagem da analise de sinais.
- O Lab RGB ajuda a conectar matematica, fisica ondulatoria e percepcao de cor.

Inventario de funcoes - Versao PC

Abaixo, as funcoes foram agrupadas por papel pedagogico e tecnico.

Inicializacao e montagem da interface

- **__init__**: inicializa frequencias, vetor de tempo, amplitudes, figura principal, estados da otimizacao e chama todas as rotinas de montagem.
 - **_setup_plots**: cria os eixos dos graficos principais, a caixa de resumo e a area de status.
 - **_update_sliders**: sincroniza visualmente os sliders com os vetores de amplitude atuais.
 - **_setup_sliders**: monta as colunas de sliders verticais para Onda 1 e Onda 2.
 - **_create_update_fn**: gera o callback de cada slider, alterando a amplitude correta e atualizando os graficos.
 - **_setup_controls**: cria os botoes Fourier, Hilbert, Lab RGB, Exercicios, Grade, OTIMIZAR, ZERAR e os sliders auxiliares.
 - **_setup_layout_grid**: desenha a grade de apoio do layout.
 - **_toggle_layout_grid**: liga ou desliga a grade de layout.
-

Controles numericos e de estado

- **_update_max_iterations**: atualiza o limite maximo de iteracoes permitido para a busca numerica.
- **_update_envelope_smoothing**: atualiza a largura da suavizacao extra aplicada aos envelopes.
- **_update_frequency_delta**: atualiza a janela espectral liberada para a otimizacao.
- **_set_status**: altera a mensagem de status mostrada ao usuario.
- **_animate_optimization_status**: gera a animacao textual do estado da otimizacao.

- **_reset_all**: zera amplitudes, limpa memoria de antes/depois e redesenha a interface.
-

Janelas teoricas e auxiliares

- **_show_exercises_tab**: abre a janela preparatoria da futura aba de exercicios da versao local.
 - **_show_fourier_before**: abre a visualizacao das equacoes antes da otimizacao.
 - **_show_fourier_after**: abre a visualizacao das equacoes depois da otimizacao.
 - **_show_fourier_equations**: organiza a janela textual das equacoes e do resumo teorico.
 - **_show_hilbert_info**: abre a janela explicativa sobre envelopes de Hilbert.
 - **_show_visible_colors**: abre o Laboratorio RGB do monitor.
-

Conversao frequencia-cor e construcao do Lab RGB

- **_frequency_to_rgb**: converte uma frequencia visivel em uma aproximacao RGB.
- **_frequency_to_wavelength_nm**: converte frequencia em comprimento de onda.
- **_clamp_to_visible_wavelength**: limita o comprimento de onda a faixa visivel usada pelo programa.
- **_compute_emitted_rgb**: calcula a cor media emitida pelas amplitudes ativas.
- **_compute_display_rgb_visual**: gera a cor visual renormalizada para comparacao na tela.
- **_compute_display_rgb_palette**: gera a cor exibida no modo paleta em RGB.
- **_normalize_rgb**: renormaliza o vetor RGB pelo canal de maior valor.
- **_frequency_to_display_rgb**: produz a cor usada para a barra do modo visual.
- **_rgb_to_hex**: converte RGB normalizado em hexadecimal.
- **_format_rgb_channels**: formata os canais RGB em texto numerico.
- **_format_rgb_compact**: produz um resumo compacto da cor em texto.
- **_draw_color_card**: desenha cada cartao de cor antes/depois para cada onda.

- **_draw_frequency_color_scale**: desenha a barra de cores por frequencia.
-

Sintese de Fourier e processamento de sinais

- **_generate_fourier_equation**: escreve a equacao simbolica da onda a partir das amplitudes ativas.
 - **_generate_wave**: monta a onda no tempo pela soma de senoides.
 - **_sanitize_smoothing_window**: garante que a janela de suavizacao seja valida e impar.
 - **_smooth_signal**: aplica media movel simples ao envelope calculado.
 - **_compute_envelope**: calcula o envelope por transformada de Hilbert e suavizacao opcional.
 - **_max_envelope_difference**: mede o erro maximo entre envelopes das duas ondas.
-

Otimizacao

- **_is_frequency_delta_unrestricted**: verifica se a janela espectral esta livre.
 - **_format_frequency_delta_label**: formata o rotulo do controle delta f.
 - **_build_optimization_mask**: escolhe quais frequencias podem variar durante a busca.
 - **_merge_optimized_values**: reconstrui os vetores completos de amplitudes a partir das componentes livres.
 - **_run_optimization**: executa o metodo L-BFGS-B para minimizar a diferenca maxima entre envelopes.
-

Atualizacao dos graficos

- **_update_plots**: recalcula ondas, envelopes, diferenca e resumo, e redesenha a tela principal.
- **_update_wave_plot**: desenha um grafico temporal de onda.

- **_update_spectrum_plot**: rotina auxiliar para grafico de espectro em barras; nao e o foco da tela atual, mas permanece como apoio tecnico.
- **_update_difference_plot**: desenha a curva delta entre os envelopes.

Versao web - app_web.py

Nesta secao, o foco recai apenas sobre as diferencas em relacao a versao PC. Os fundamentos matematicos da sintese de Fourier, dos envelopes de Hilbert, do criterio de semelhanca entre envelopes e do Laboratorio RGB permanecem conceitualmente identicos; por isso, nao sao repetidos em detalhe aqui.

1. Diferencas de arquitetura e de interface

A principal alteracao introduzida em **app_web.py** diz respeito ao paradigma de interface. Em vez de uma figura unica do Matplotlib acompanhada de janelas auxiliares, a versao web organiza a experiencia em abas permanentes, atualizadas por callbacks reativos do Dash. Esse redesenho modifica o modo de navegacao do estudante: em lugar de abrir janelas independentes, o usuario percorre uma interface unica, organizada em blocos tematicos.

Aba Simulador

Substitui a grande figura da versao PC por um painel web com tabela editavel, graficos Plotly, controles de parametros e vistas adaptadas para celular.

Abas teoricas permanentes

Fourier Antes, Fourier Depois, Hilbert e Lab RGB deixam de ser janelas separadas e passam a compor a navegacao principal da pagina.

Persistencia local

Estados do simulador, historico e progresso de exercicios podem ser mantidos no navegador, o que nao existia na versao local.

2. Diferencas na tela Simulador

A tela Simulador conserva os mesmos objetos conceituais da versao PC, mas muda a forma de manipulacao. Os sliders verticais extensos sao substituidos, no desktop, por uma tabela editavel de amplitudes e, no celular, por um editor de uma frequencia por vez. O grafico espectral por barras, construído em **build_intensity_figure**, ganha maior centralidade porque passa a funcionar como ponte visual entre a tabela numerica e a resposta cromatica do sistema.

Outro ponto distintivo e a figura principal 2x2 produzida por **build_main_figure**. Embora ela mantenha as curvas ja presentes na versao PC, sua implementacao com Plotly permite uma composicao mais integrada para navegacao em navegador. Alem disso, a funcao **lock_figure_interaction** restringe gestos de zoom e arrasto, privilegiando a leitura guiada em contexto didatico.

3. Diferencas no tratamento pedagogico

A maior inovacao da versao web e a presenca de uma sessao estruturada de exercicios. Na versao PC, a area de exercicios ainda aparece como esboco conceitual. Em **app_web.py**, por outro lado, o estudante percorre uma sequencia de etapas com objetivos distintos, produz respostas curtas, recebe feedback automatico, escreve uma conclusao final e envia um relatorio consolidado.

Diferenca pedagogica decisiva: a versao web deixa de ser apenas um ambiente de exploracao aberta e passa a operar tambem como instrumento de avaliacao formativa, orientacao passo a passo e registro institucional da atividade.

As etapas implementadas em **generate_random_exercise** exploram metas de aproximacao entre envelopes, observacao do efeito da suavizacao, analise da restricao espectral por Δf e comparacao entre envelope e representacao cromatica. Em outras palavras, o simulador deixa de apenas mostrar fenomenos e passa a solicitar interpretacoes sobre esses fenomenos.

4. Diferencas na camada de dados e acompanhamento

A versao web tambem acrescenta uma camada de dados inexistente na implementacao local. O arquivo **alunos.js** fornece trilhas, turmas e nomes; as funcoes de integracao com Google Sheets gravam resultados finais; e a rotina **send_final_session_to_class_worksheet** atualiza automaticamente a aba

correspondente da turma. Assim, a simulacao passa a ocupar tambem um lugar administrativo no fluxo de avaliacao.

Essa mudanca e relevante do ponto de vista de um artigo de ensino porque transforma o simulador em um artefato que nao apenas media a aprendizagem, mas produz rastros avaliativos, historico de desempenho e evidencias escritas de compreensao conceitual.

5. Diferencas no Laboratorio RGB e no uso em dispositivos moveis

Embora o Laboratorio RGB mantenha a mesma base conceitual da versao PC, a versao web o integra com mais clareza ao fluxo geral da atividade: ha atualizacao automatica dos cartoes, seletor explicito entre modos de exibicao e adaptacao para telas pequenas. O mesmo vale para o conjunto do simulador, que passa a dispor de resumos moveis, navegacao entre frequencias e composicoes visuais mais compactas.

6. Relevancia academica da versao web

Do ponto de vista de um texto academico, a versao web representa uma transicao de um *simulador exploratorio local* para uma *plataforma didatica distribuida*. A base teorica permanece, mas a mediação pedagogica se amplia: ha orientacao de percurso, responsividade para celular, acompanhamento de respostas, consolidacao de resultados e integracao com instrumentos de registro escolar. Essa ampliacao sugere uma passagem do uso demonstrativo para o uso sistematico em sequencias de ensino.

Inventario de funcoes - Versao web

Como o arquivo web e bem maior, as funcoes foram separadas por familias: nucleo numerico, integracao de dados, renderizacao visual, exercicios e callbacks.

Nucleo numerico e representacao dos sinais

- **clamp_amp**: limita amplitudes ao intervalo permitido.
- **amps_to_table_data**: converte vetores de amplitude em linhas da tabela editavel.
- **table_data_to_amps**: reconstrui os vetores de amplitude a partir da tabela.
- **generate_wave**: gera a onda no tempo pela soma de senoides, usando a base precomputada.
- **sanitize_smoothing_window**: valida a janela de suavizacao.
- **smooth_signal**: aplica media movel ao envelope.
- **optimization_smoothing_window**: escolhe a janela de suavizacao usada durante a busca numerica.
- **compute_envelope**: calcula envelope por Hilbert e suavizacao opcional.
- **max_envelope_difference**: calcula o erro maximo entre envelopes com a leitura final exibida ao usuario.
- **max_envelope_difference_fast**: calcula uma versao otimizada do erro para a rotina de busca.
- **is_frequency_delta_unrestricted**: verifica se a janela espectral esta livre.
- **format_frequency_delta**: formata o texto do delta f.
- **build_single_wave_optimization_mask**: monta a mascara de frequencias livres para uma unica onda.
- **build_optimization_masks**: combina as mascaras das duas ondas.
- **merge_optimized_values**: reinsere as amplitudes livres dentro dos vetores completos.
- **normalize_text**: normaliza texto para comparacoes e busca de palavras-chave.

Banco de estudantes e integracao com planilhas

- **infer_grade_class_from_track**: tenta inferir serie e turma a partir do nome da trilha.
- **load_student_database**: le o arquivo de alunos usado na versao web.
- **build_track_options**: monta as opcoes de trilha.
- **build_class_options**: monta as opcoes de turma.
- **build_student_name_options**: monta a lista de estudantes coerente com trilha, serie e turma.
- **serialize_json_value**: serializa estruturas para envio HTTP.
- **is_google_sheets_configured**: verifica se as credenciais da planilha existem.
- **parse_service_account_json**: interpreta JSON cru, serializado ou em base64.
- **load_service_account_info**: carrega as credenciais da conta de servico.
- **get_google_sheets_client**: cria o cliente autenticado do Google Sheets.
- **open_google_spreadsheet**: abre a planilha por ID ou nome.
- **get_google_worksheet_with_headers**: abre ou cria uma aba e garante o cabeçalho.
- **get_google_spreadsheet**: devolve a planilha principal configurada.
- **get_google_worksheet**: abre a aba usada para registros brutos do app web.
- **get_google_final_results_worksheet**: abre a aba de resultados finais da sessao.
- **send_submission_to_google_sheets**: envia um registro bruto de exercicio para a planilha.
- **send_final_session_to_google_sheets**: envia o relatorio final da sessao para a aba de resultados.

Atualizacao da aba da turma

- **parse_stage_details**: le o JSON com os detalhes das etapas resolvidas.
- **compute_stage_average_100**: calcula a media das etapas em escala de 0 a 100.
- **build_group_name**: monta o nome da turma combinando serie e turma.

- **sanitize_worksheet_title**: limpa o titulo da aba para o padrao do Google Sheets.
- **format_grade_10**: converte a nota final para a forma textual gravada na aba.
- **is_bimester_label**: identifica rotulos de bimestre.
- **column_index_to_letter**: converte indice numerico em letra de coluna.
- **ensure_worksheet_size**: garante quantidade minima de linhas e colunas na aba.
- **get_or_create_class_worksheet**: abre ou cria a aba da turma.
- **get_class_header_rows**: le e valida as duas linhas de cabecalho da aba da turma.
- **build_class_student_row_map**: mapeia nome do estudante para numero da linha.
- **append_missing_class_students**: adiciona estudantes ausentes ao final da aba.
- **find_bimester_start**: encontra a coluna onde comeca um bimestre.
- **get_next_bimester_start**: encontra o proximo bloco de bimestre.
- **ensure_bimester_and_simulation_column**: cria ou reutiliza a coluna Soma de Cores dentro do bimestre correto.
- **update_class_simulation_column**: grava as notas da simulacao na coluna correta.
- **update_class_bimester_total_column**: reconstroi a formula de total do bimestre.
- **write_class_sheet_worksheet**: coordena a atualizacao segura da aba da turma inteira, sem apagar outras simulacoes.
- **build_class_sheet_summary_row**: monta a linha-resumo minima usada na sincronizacao da turma.
- **send_final_session_to_class_worksheet**: liga o envio final do estudante a atualizacao automatica da aba da turma.

Envio final, relatorio e mensagens

- **post_json_to_apps_script**: envia JSON para os endpoints externos usados pelo projeto.
- **build_final_session_payload**: consolida todos os dados finais do estudante, da sessao e das etapas.
- **send_final_session_results**: coordena planilha, aba da turma, e-mails e mensagem final de sucesso ou erro.

- **append_feedback_detail**: acrescenta detalhes a um bloco de feedback.
 - **build_exercise_submission**: monta um registro detalhado de submissao de etapa.
-

Fourier, RGB e visualizacao principal

- **generate_fourier_equation**: transforma as amplitudes ativas em expressao textual da sintese de Fourier.
- **frequency_to_wavelength_nm**: converte frequencia em comprimento de onda.
- **clamp_to_visible_wavelength**: prende o valor a faixa visivel tratada pela simulacao.
- **frequency_to_rgb**: traduz frequencia em cor RGB aproximada.
- **normalize_rgb**: renormaliza o vetor RGB.
- **compute_emitted_rgb**: calcula a emissao RGB media gerada pelas amplitudes.
- **compute_display_rgb_visual**: produz a cor final do modo RGB visual.
- **compute_display_rgb_palette**: produz a cor final do modo paleta em RGB.
- **rgb_to_hex**: transforma RGB em cor hexadecimal.
- **format_rgb**: formata os canais RGB como texto.
- **has_valid_optimization_result**: verifica se existe um antes/depois coerente de otimizacao.
- **lock_figure_interaction**: trava zoom e pan para manter a leitura guiada.
- **build_main_figure**: monta a figura principal 2x2 com ondas, envelopes e diferenca.
- **build_intensity_figure**: monta os graficos de barras das amplitudes por frequencia.
- **build_summary**: cria o resumo textual do estado atual da simulacao.
- **build_color_scale_figure**: monta a barra de cores por frequencia.
- **build_rgb_card**: cria um cartao RGB informativo para uma onda em um certo estagio.
- **build_pending_rgb_card**: cria o cartao de espera para o caso de nao haver resultado apos otimizacao.
- **optimize_amplitudes**: executa a busca numerica da versao web.
- **build_hilbert_notes**: gera os textos didaticos dinâmicos sobre suavizacao e delta f.

Sessao de exercicios

- **generate_random_exercise**: sorteia um exercicio de um tipo especifico ou geral.
- **build_exercise_rubric**: monta a lista de criterios de conferência.
- **build_exercise_feedback**: desenha a caixa de feedback pedagogico.
- **create_exercise_session**: cria uma sessao completa com quatro etapas.
- **normalize_exercise_session**: valida e normaliza o estado salvo da sessao.
- **get_current_session_exercise**: devolve a etapa corrente da sessao.
- **build_stage_rubric**: escolhe os criterios apropriados para etapa, conclusao ou resultados.
- **build_session_results_summary**: resume a sessao inteira em numeros e percentual final.
- **build_exercise_stage_outputs**: coordena o que aparece em cada estagio da aba Exercicios.
- **build_exercise_progress_panel**: mostra historico, precisao, sequencias e placar recente.
- **collect_simulation_metrics**: extrai metricas numericas da simulacao para avaliacao automatica.
- **evaluate_exercise**: decide acerto, parcial ou erro combinando parte numerica e parte textual.
- **build_mobile_active_summary**: resume no celular quais frequencias estao ativas.
- **clone_figure_for_mobile**: adapta um grafico para leitura em telas pequenas.
- **build_mobile_visual_content**: organiza o bloco visual do modo movel.

Callbacks principais do Dash

- **handle_actions**: responde a otimizar, zerar e aplicar alteracao movel.
- **refresh_lab_rgb**: atualiza a mensagem de horario do laboratorio RGB.

- **refresh_outputs**: recalcula tudo que aparece nas abas Fourier, Hilbert, Lab RGB e Simulador.
 - **browse_mobile_frequency**: navega entre frequencias no editor movel.
 - **sync_mobile_editor**: sincroniza os campos moveis com a frequencia selecionada.
 - **handle_exercises**: coordena o fluxo inteiro da sessao de exercicios.
 - **sync_student_group_with_track**: preenche serie e turma a partir da trilha quando possivel.
 - **sync_student_name_options**: atualiza os nomes disponiveis conforme a selecao atual.
 - **handle_final_results_send**: valida os campos finais, monta o payload e dispara o envio definitivo.
-

Fechamento didatico

As duas versoes mostram a mesma ideia fisica e matematica por caminhos de interface diferentes. A versao PC e excelente para exploracao local e demonstracao ao vivo. A versao web amplia o alcance: roda no navegador, adapta-se ao celular, registra estudantes, cria exercicios e transforma o simulador em uma plataforma completa de atividade guiada.